

Trabalho prático final

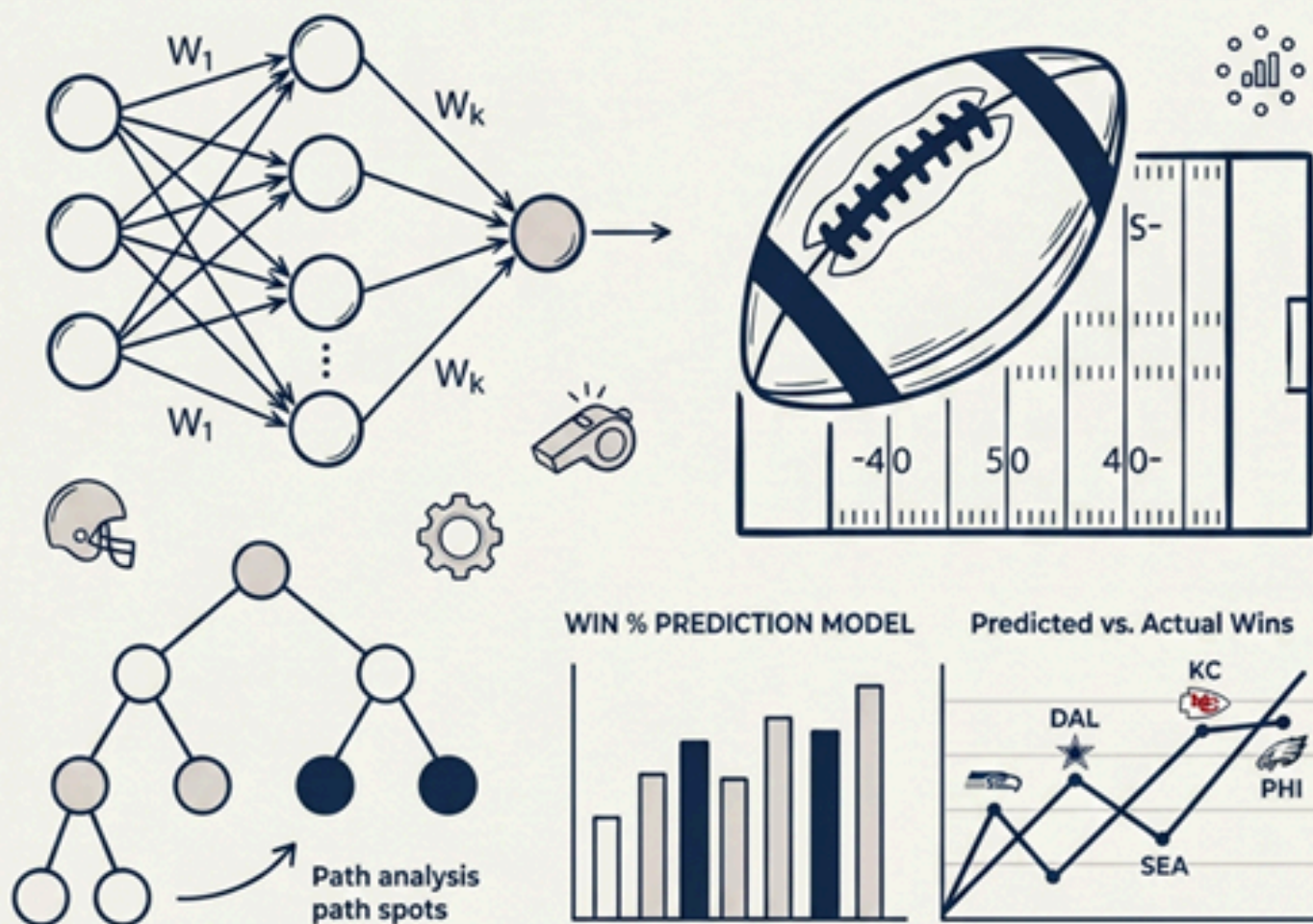
Modelagem preditiva com Aprendizado de Máquina: Caracterização do problema e *spot-checking* de algoritmos

Lucas Lemos Merotto - 00166157

Pedro Simões Bonjardim - 00335130

INF01017 - Aprendizado de Máquina

Profa. Mariana Recamonde Mendoza



1. Coleta de dados e identificação do problema

No filme *Moneyball* - O Homem que Mudou o Jogo (2011), Billy Beane (personagem interpretado por Brad Pitt) começa a implementar um modelo inovador de tomada de decisões no time de beisebol gerido por ele, o Oakland Athletics. Em parceria com seu jovem colega Peter Brand, os dois começam a aplicar equações simples calculadas por computador para contratar jogadores outrora desvalorizados. Os algoritmos utilizados por eles ignoravam atributos insignificantes para a habilidade de um jogador: idade, aparência, personalidade. Ao invés, eles selecionavam os jogadores mais subestimados, que normalmente seriam desprezados pelos gerentes tradicionais do esporte, inclusive aqueles dentro do próprio Oakland A's.

Nesse filme baseado em uma história real, pode-se testemunhar uma das primeiras aplicações da computação ao esporte. Fato que era encarado com muita estranheza à época da trama (ano de 2002), hoje já é rotineiro para a imensa maioria dos times das principais ligas esportivas do mundo. Com o advento dos algoritmos de aprendizado de máquina estudados na disciplina, a previsão esportiva se tornou ainda mais precisa, sendo que as equipes estão usando-os cada vez mais. Fato é que o Oakland A's de Brad Pitt conseguiu vencer 20 jogos consecutivos a partir de um simples programa em R. Já pensou no que poderíamos fazer com uma rede neural?

Foi a partir desse ponto que a dupla escolheu trabalhar com um problema esportivo, mais especificamente relacionado ao futebol americano. A National Football League (NFL) é a principal liga de futebol americano do mundo, composta por 32 times (que são fixos, não havendo rebaixamento ou acesso de séries inferiores como no Campeonato Brasileiro). Cada temporada regular (que todos os times participam, antes dos *playoffs* ou “mata-mata”) inicia em setembro e se estende até janeiro do ano seguinte. Cada um dos 32 times jogam 16 jogos na temporada regular (apesar desse número aumentar para 17 a partir de 2021, como trataremos em Análise exploratória e pré-processamento dos dados).

Em uma partida de futebol americano, ganha o time que somar mais pontos ao final do tempo regulamentar e, se necessário, prorrogação (raramente havendo empate). No entanto, não é apenas o número de pontos marcados que indica o sucesso (ou fracasso) de um time ao longo de uma temporada. Dados como jardas avançadas, jogadas ofensivas, *turnovers* (perda da posse de bola), *touchdowns* corridos e aéreos podem nos dizer muito sobre a “saúde” de uma equipe de futebol americano.

Assim, nosso trabalho tem como objetivo desenvolver um modelo baseado em Aprendizado de Máquina para prever a porcentagem de vitórias de um determinado time da NFL em uma determinada temporada, a partir de estatísticas acumuladas ao longo dos jogos desta temporada. Com esse objetivo, utilizamos o *dataset* “NFL Team Data 2003-2023” disponível em <https://www.kaggle.com/datasets/nickcantalupa/nfl-team-data-2003-2023>

2. Análise exploratória e pré-processamento dos dados

A partir do *dataset* selecionado, a dupla buscou identificar quais atributos seriam relevantes para a predição dos modelos, quais poderiam ou deveriam ser descartados e aqueles que precisam de manipulação prévia para serem usados como dado. O primeiro desafio encontrado foi para escolher o atributo de predição, isso porque o número de vitórias em uma temporada não se mostrou um bom parâmetro. Isso ocorreu pois em 2021 a organização da NFL acrescentou um jogo a mais na temporada regular, passando de 16 para 17 confrontos. Isso poderia interferir no resultado, pois teríamos escalas diferentes para cada instância. Dessa forma, optou-se pelo percentual de vitórias na temporada regular, sendo uma métrica mais precisa e justa para todas as instâncias. O *dataset* original possui 35 atributos, mas alguns deles foram excluídos:

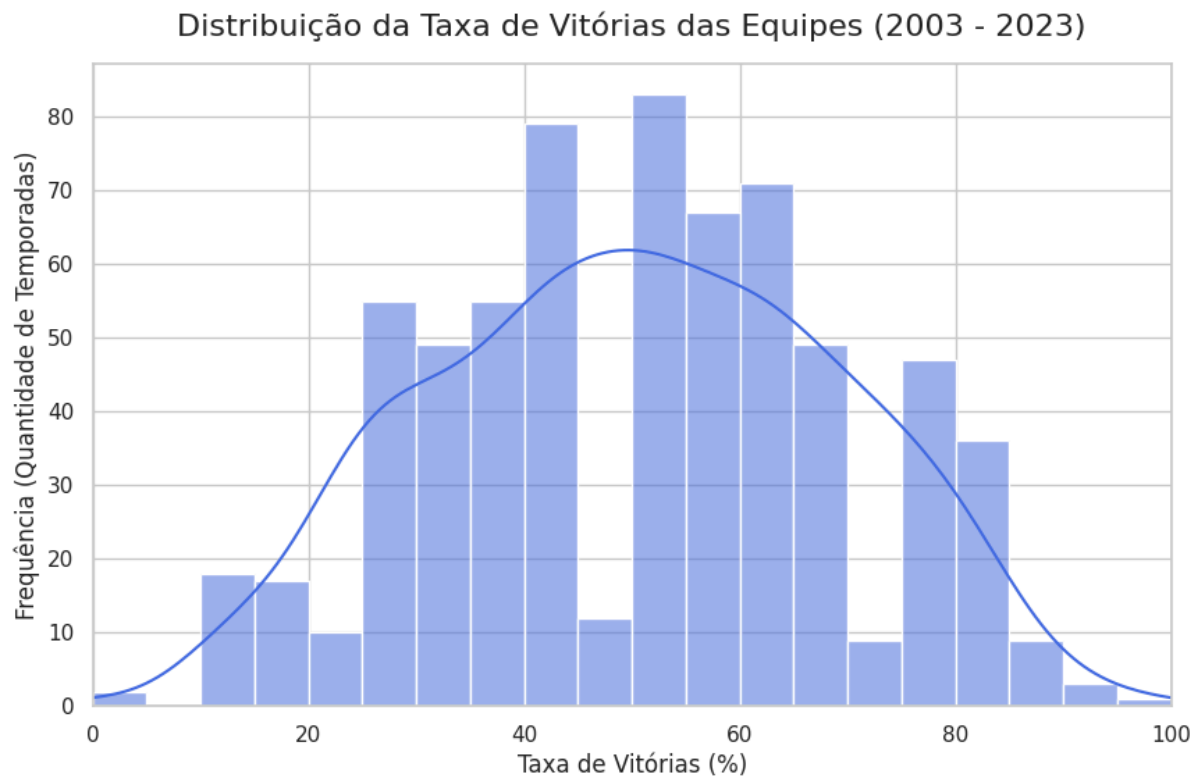
- Número de vitórias e número de derrotas, que já está implícito no atributo de percentual de vitórias;
- Diferença média de pontos por partida, pois cerca de metade das instâncias estava com esse atributo faltante;

- Pontos esperados na temporada, por se tratar de uma métrica derivada de uma função que não foi apresentada.

Sendo assim, restaram 31 atributos, sendo que o único atributo nominal é o nome do time e os demais, numéricos, contemplando todas as estatísticas ofensivas de uma instância durante a temporada regular. Nelas estão a quantidade de pontos por jogadas aéreas, terrestres, jogadas especiais, quantidade de jardas avançadas, erros cometidos no ataque e outras métricas que são comuns nas estatísticas do esporte.

O atributo que evidenciava o ano da temporada foi removido, devido ao fato de que a dupla não acreditava apresentar relevância para a resolução do problema. Todavia, foi acrescentada uma nova *feature* que não compunha o *dataset* original: porcentagem de vitórias nas últimas três temporadas, sendo esses dados coletados através da média de vitórias dos anos anteriores disponíveis no *dataset*. Quando precisou de dados que estavam fora do escopo temporal do *dataset* (antes de 2003), houve a coleta manual dos dados através das estatísticas oficiais da NFL. Essa métrica pode ser valiosa para o aprendizado do modelo, pois é possível que um time que estava bem repita um desempenho satisfatório no próximo ano, valendo analogamente para uma equipe que estava mal, e continue não empolgando.

O atributo alvo intrinsecamente apresenta uma distribuição que evita altas concentrações nos extremos (comportamento semelhante a uma distribuição normal), uma vez que a vitória de um time é sempre a derrota de outro. Sendo assim, a soma total das porcentagens é muito próxima da metade do número total de instâncias, que apenas não é o valor exato pois há algumas partidas que terminam em empate, sendo apenas 13 em um total de 5.424. A partir do gráfico abaixo, é possível perceber que a maioria das equipes apresentam uma taxa de vitórias próxima aos 50%:



O processo de pré-processamento de dados envolveu utilizar algumas técnicas que aprendemos durante o semestre na disciplina de Aprendizado de Máquina. Como a maioria das *features* do *dataset* escolhido são numéricas, foi utilizada a sua normalização. Isso é importante, pois muitos modelos utilizados no processo de *spot-checking* são sensíveis à normalização dos dados, ou seja, um atributo em maior escala pode influenciar mais na decisão do modelo de forma indevida, simplesmente por conter números de grandezas maiores. A normalização aplicada nos dados foi a *MinMaxScaler*, onde define 0 para o valor mais baixo da *feature* e 1 para o mais alto, e posiciona os demais dentro desse intervalo, respeitando a proporcionalidade deles.

Para o atributo do nome de cada time, por ser nominal, foi utilizado o *OneHotEncoder*, onde define uma tabela em que cada time só recebe 1 quando o seu nome for correspondente, sendo assim, cada um dos 32 times terá um único atributo definido em 1 e os outros trinta e um atributos em 0. Isso é necessário para que o modelo não infira nenhuma ordem de grandeza entre os diferentes times,

como o que poderia acontecer com o *label*. Esse atributo também rendeu uma modificação manual, pois alguns times mudaram de nome nesse intervalo de 20 anos. Isso ocorre tanto por questão de marketing, ou também devido a mudança da cidade-sede, geralmente por motivos financeiros.

Por fim, foi aplicada a estratificação, para garantir que as classes majoritárias não dominem o grupo de treinamento, mantendo sempre a proporcionalidade entre conjunto de treino e teste. Para isso ser feito, seria uma complicação ter apenas um representante de um determinado atributo alvo, pois ele estaria ou apenas no teste ou apenas no treino. Por esse motivo, foram removidas as instâncias com apenas um exemplar no atributo alvo, por serem consideradas outliers. Além disso, não foi uma grande perda para o poder do modelo, pois foram removidas apenas 5 instâncias, reduzindo o conjunto completo de 672 para 667 instâncias.

3. Definição da abordagem, dos algoritmos, e da estratégia de avaliação

O objetivo de prever o percentual de vitórias de um time na temporada regular, a partir de suas estatísticas ofensivas, se caracteriza por produzir uma saída numérica. Dessa maneira, se configura como um problema de regressão, logo os modelos escolhidos seguiram o norte de serem regressores. Buscou-se utilizar os principais algoritmos mostrados em aula, cada um visto em uma aula diferente da disciplina. Outra meta no momento da escolha foi representar tanto modelos que priorizam viés em detrimento da variância quanto modelos que priorizam variância em detrimento do viés, com o intuito de trazer maior diversidade e uma melhor base para o *spot-checking*.

Os algoritmos escolhidos foram:

- 1. KNN (K-vizinhos mais próximos)**
- 2. Árvore de Decisão**
- 3. Regressão Linear**
- 4. Redes Neurais**
- 5. Floresta Aleatória**

Todos eles foram aplicados com hiperparâmetros padrões, ou com uma pequena variação. Para a avaliação dos modelos criados, foram selecionados os dois principais métodos utilizados para problemas de regressão. São eles o erro médio absoluto (MAE) e o erro médio quadrático (MSE). A primeira abordagem faz um panorama geral do quanto o modelo está errando para cada instância de teste, na média. Já a diferença para a segunda métrica é que nela os erros são elevados ao quadrado, punindo de forma mais rigorosa grandes erros. Isso dá um panorama diferente, pois é possível avaliar se são erros menores, porém com maior frequência, ou se são erros mais raros, mas que têm uma margem de erro maior.

4. Spot-checking de algoritmos

Para a divisão dos dados em conjunto de treino e teste, foi utilizado o método de *k-fold cross validation*, com 5 divisões, pois se caracteriza como uma ótima alternativa para evitar o *overfitting*. Essa estratégia garante que cada dado irá passar pelo conjunto de teste uma única vez, evitando a surpresa de ele nunca ter visto esse dado anteriormente. Outro ponto que esse método leva vantagem é pelo motivo de que, ao criar múltiplas divisões de treino e teste, evita que o modelo apenas decore os dados de uma única divisão, o que pode acontecer com o *holdout*, dessa forma, reduzindo o *overfitting*.

A maior parte do *spot-checking* de algoritmos pode ser conferido no arquivo .ipynb disponibilizado pela dupla, mas consiste nas etapas explicitadas nas seções anteriores. Após carregamento dos dados, eles foram divididos entre treino e teste, e pré-processados em cada fold, utilizando normalização de atributos numéricos e codificação de atributos categóricos. Depois foi feito um *for*, onde a cada iteração, os dados eram utilizados para treinar e testar um dos algoritmos escolhidos e deles foram extraídas as métricas de avaliação, erro médio absoluto e erro médio quadrático.

5. Sumarização de Resultados - Spot-checking

Por se tratar de um problema de regressão, as principais métricas são baseadas no erro, ou seja, a diferença entre a porcentagem de vitórias predita pelo modelo e a real. Ademais, podemos afirmar que a *random forest* e regressão linear foram os dois modelos mais precisos, com MAEs próximos de 0,068 e 0,070, indicando que tanto modelos baseados em árvores quanto lineares capturaram a complexidade do problema de forma precisa. Essa magnitude de erro, no escopo de 16 ou 17 jogos, representa algo próximo a uma vitória, tendo os melhores modelos uma margem de erro de apenas uma vitória para mais ou para menos.

Ainda é possível observar que os testes feitos com os atributos (como adição da média de desempenho das últimas três temporadas ou *OneHotEncoding* do nome dos times) afetam menos a regressão linear, demonstrando a robustez do modelo frente aos dados fornecidos. Sobre a rede neural, embora tenha ficado atrás dos outros dois algoritmos já citados, teve um desempenho satisfatório. Vale discutir que para este volume de dados, este modelo mais complexo não superou os tradicionais.

Outro ponto interessante é observado através do erro médio quadrático, que está relativamente baixo para todos os modelos testados. Isso significa que quando esses modelos erram suas predições, não estão sendo erros absurdos. Ou seja, mesmo não acertando a porcentagem de vitórias de um time, eles conseguem captar muito bem a situação da equipe, podendo não acertar o número exato, mas tendo acertado se o time teve uma temporada boa, mediana ou ruim. Nessa métrica, novamente a regressão linear e a floresta aleatória foram os modelos que mais se saíram bem. Isso mostra que além de terem uma taxa de acerto satisfatória, quando erram, erram por pouco.

Continuando a reflexão sobre *random forest*, podemos justificar a escolha por otimizar esse modelo na próxima fase do trabalho devido ao seu equilíbrio entre precisão e capacidade de capturar relações não-lineares. Fica como ponto focal para a próxima etapa do desenvolvimento do trabalho uma otimização de hiperparâmetros. A floresta aleatória sem poda teve um desempenho levemente melhor, ao compararmos com a podada com profundidade 10. É muito possível, então, que o número ideal não foi atingido para equilibrar o *overfitting* e a precisão.

Da mesma forma, é pertinente otimizar a Regressão Linear, um *baseline* sólido e interpretável.

Box plot comparando os modelos quanto a Erro Absoluto Médio (MAE):

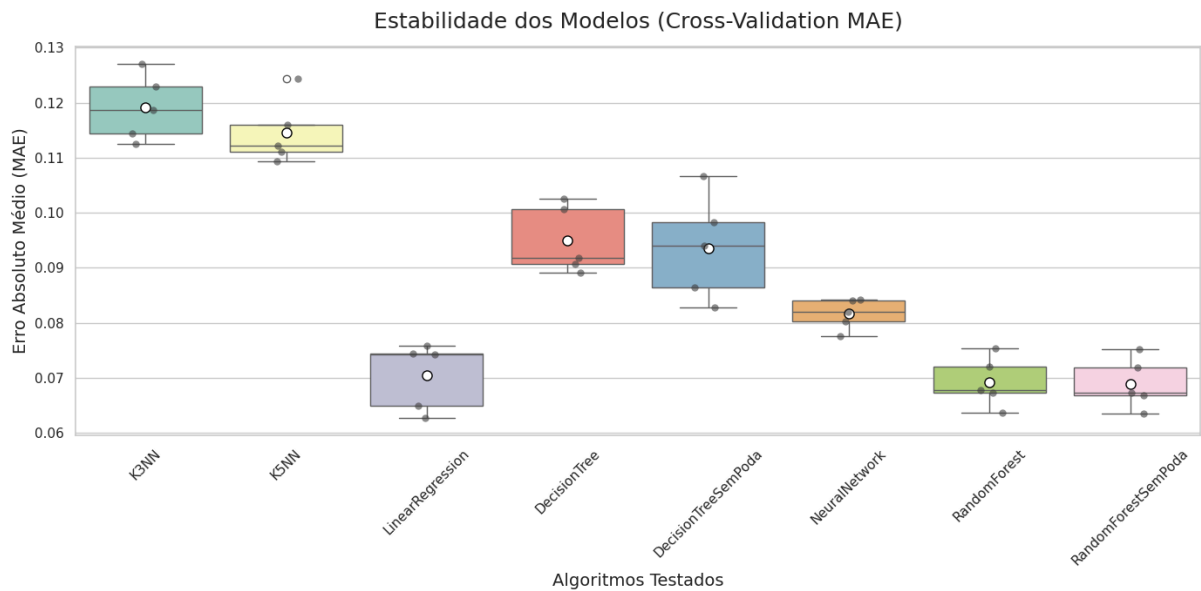


Tabela com a MAE e MSE de cada algoritmo em 5 diferentes *random states* e seu resultado médio:

	rs=41	rs=42	rs=43	rs=44	rs=45	média
knn3						
mae	0,1168	0,1201	0,1157	0,1194	0,1191	0,11822
mse	0,0215	0,0223	0,021	0,0221	0,0224	0,02186
knn5						
mae	0,1156	0,1175	0,113	0,1218	0,1146	0,1165
mse	0,0209	0,0212	0,0202	0,0223	0,0211	0,02114
regressao						
mae	0,0678	0,0699	0,0691	0,0738	0,0704	0,0702
mse	0,0081	0,0084	0,0082	0,0091	0,0085	0,00846
tree						
mae	0,091	0,0933	0,0893	0,0948	0,095	0,09268
mse	0,0144	0,0151	0,0153	0,0151	0,0154	0,01506
treeSemPoda						
mae	0,0923	0,0939	0,0913	0,0995	0,0936	0,09412
mse	0,015	0,0154	0,0158	0,0164	0,0156	0,01564
redeNeural						
mae	0,0797	0,0771	0,0797	0,0786	0,0816	0,07934
mse	0,0106	0,0099	0,0107	0,0101	0,011	0,01046
randomForest						
mae	0,0683	0,0709	0,0672	0,071	0,0692	0,06932
mse	0,0081	0,0086	0,008	0,0085	0,0081	0,00826
rfSemPoda						
mae	0,0686	0,0711	0,0672	0,0712	0,0689	0,0694
mse	0,0082	0,0086	0,008	0,0085	0,0081	0,00828

6. Otimização de Hiperparâmetros

Para a segunda etapa do trabalho da disciplina, os algoritmos selecionados pela dupla para um maior detalhamento e exploração dos seus hiperparâmetros foram a regressão linear e a floresta aleatória. O método selecionado foi o *holdout* com validação cruzada aninhada com 5 divisões. Esse método é muito interessante por conseguir trabalhar com uma maior variabilidade de dados, e garantir que todas as instâncias estejam presentes no conjunto de teste por uma única vez. O lado negativo é o tempo de processamento, pois ele roda cada configuração diferente de modelo 5 vezes, levando um tempo de processamento bem mais elevado que o *holdout* de 3 vias. Entretanto, por ser um problema com, relativamente, poucas instâncias, a dupla acredita que esse *tradeoff* valha a pena.

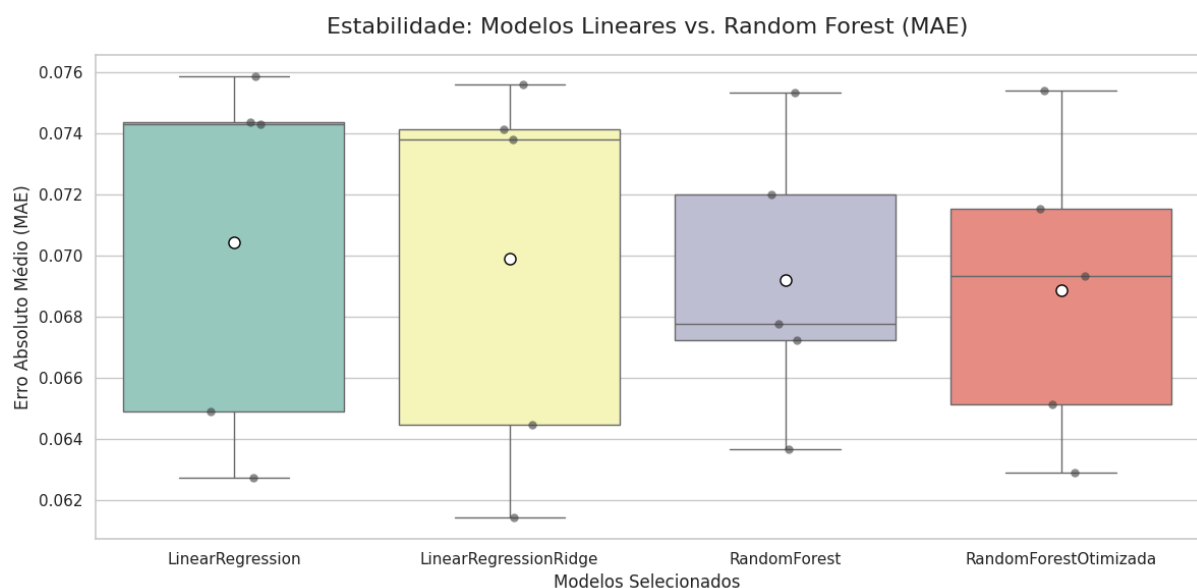
A regressão linear não é um modelo complexo, ela é simples o bastante para não haver hiperparâmetros. Entretanto, existe a Regressão Ridge, que possui o mesmo fundamento da Regressão Linear, todavia acrescenta um hiperparâmetro *alpha* que representa a penalização que o algoritmo sofre ao errar um resultado. É uma técnica bem efetiva a fim de minimizar o *overfitting*, que pode acabar por errar as previsões com dados totalmente novos. Foram rodados diversos valores do parâmetro, que simboliza o peso da penalização, além disso, foram utilizados 5 *K's* para a validação cruzada. A otimização deste hiperparâmetro foi atingida com um *alpha* de 0.04.

A floresta aleatória é um algoritmo de uma maior complexidade, e que, por sua vez, apresenta vários hiperparâmetros, os quais para otimizar, foi necessário encontrar a melhor combinação entre eles. Primeiramente, há o número de árvores que o modelo vai utilizar, foi identificado que os melhores resultados foram obtidos com valores próximos de 200 árvores. Dentre estes, o número de 190 árvores foi o que apresentou o melhor desempenho entre os modelos analisados. Outro hiperparâmetro muito importante para o algoritmo é a questão da pré-poda, ou seja, já definir um limite de profundidade máxima que as árvores poderão ter. Foram rodados testes com valores de 1 a 20, crescendo de 5 em 5, além da possibilidade de não haver poda. O melhor resultado foi 5, então analisamos com valores entre 3 e 7, e após rodar os modelos, o melhor resultado foi aquele que apresenta árvores com profundidade máxima de 6 níveis. A mesma lógica foi seguida com outros 2

hiperparâmetros manipulados, que definem o número mínimo de amostras necessárias para dividir um nodo, e também o número mínimo de amostras para um nodo ser uma folha. Os melhores valores encontrados para esses hiperparâmetros foram 5 e 2, respectivamente.

7. Avaliação de desempenho

Repetindo as mesmas métricas utilizadas anteriormente para avaliar os diferentes algoritmos na fase de *spot-checking*, foi possível perceber a evolução dos modelos com a otimização dos hiperparâmetros. Embora a diferença numericamente tenha sido pequena entre os modelos originais e os otimizados, em todos os *random states* houve melhorias nos resultados com a otimização. A regressão com o parâmetro *alpha* apresentou uma redução de 0,04% no erro médio absoluto nas suas previsões, enquanto o modelo de floresta aleatória foi aprimorado em 0,06% na mesma métrica. Do mesmo modo, a regressão linear, através da otimização de hiperparâmetros, reduziu o erro médio quadrático em 0,01%, enquanto a floresta aleatória diminuiu em 0,03%.



	rs=41	rs=42	rs=43	rs=44	rs=45	média	melhoria
randomForest - Sem otimização							
mae	0,0683	0,0709	0,0672	0,071	0,0692	0,0693	
mse	0,0081	0,0086	0,008	0,0085	0,0081	0,0083	
randomForest - Otimizada							
mae	0,0673	0,0701	0,0666	0,0708	0,0689	0,0687	0,06%
mse	0,0078	0,0082	0,0077	0,0083	0,008	0,008	0,03%
regressao - Sem otimização							
mae	0,0678	0,0699	0,0691	0,0738	0,0704	0,0702	
mse	0,0081	0,0084	0,0082	0,0091	0,0085	0,0085	
regressao - Otimizada							
mae	0,0676	0,0697	0,0685	0,0732	0,0699	0,0698	0,04%
mse	0,008	0,0083	0,0081	0,009	0,0084	0,0084	0,0001

Dessa maneira, a floresta aleatória, que mesmo sem otimização de hiperparâmetros, já era o modelo mais promissor do *spot-checking* de algoritmos, como obteve uma melhoria maior que a da regressão linear quando houve a manipulação dos hiperparâmetros, ampliou sua vantagem frente ao modelo concorrente. Sendo assim, mesmo sendo modelos de assertividade semelhantes, para nosso modelo final foi escolhida a floresta aleatória otimizada.

Por fim, após identificação de que o ajuste fino do modelo já foi realizado, a última etapa é utilizar os dados de teste globais, que foram separados no início da construção do modelo. Esses dados não foram vistos em nenhum momento pelo modelo, nem na etapa de treino e nem nos testes dentro da validação cruzada aninhada.

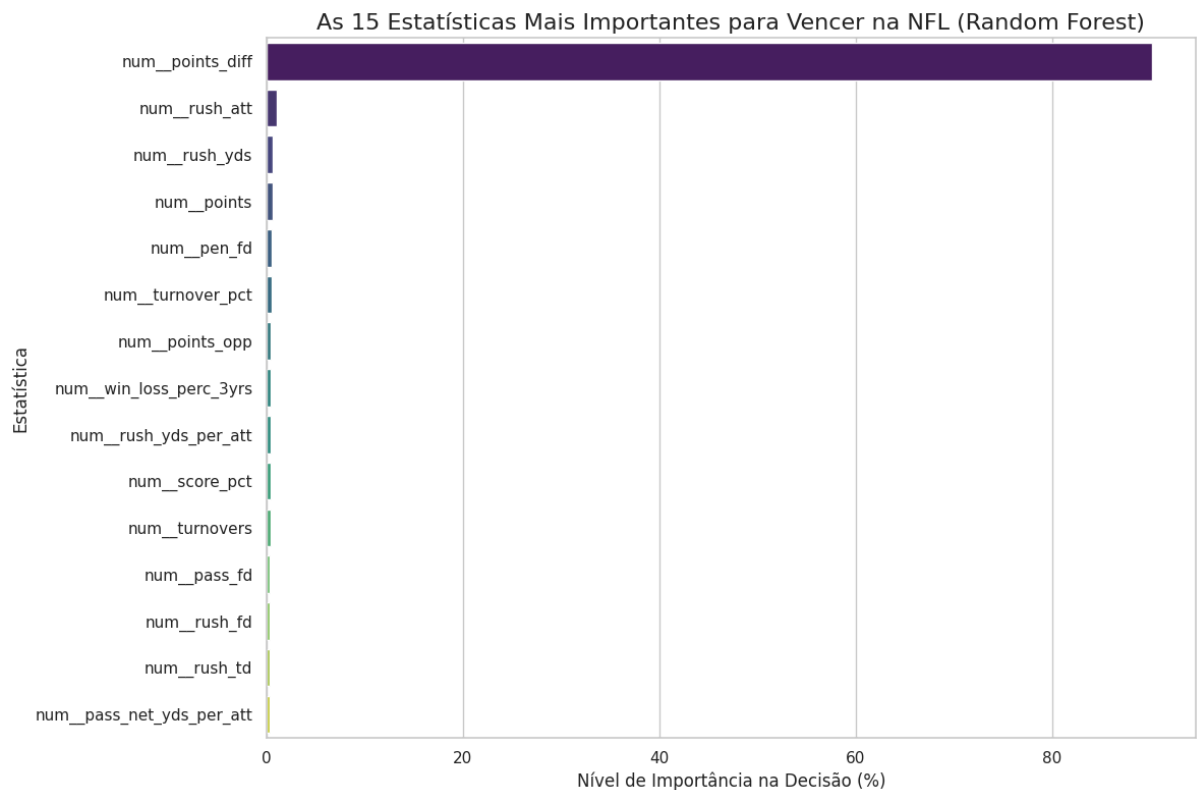
O resultado foi muito satisfatório, pois não indicou *overfitting* do modelo nos dados de treino, a piora de desempenho foi mínima, ao analisarmos a média de erro absoluto, 0,03%. O erro quadrático médio teve um desempenho ainda melhor, pois houve uma melhoria de 0,01% nos dados de teste. Dessa maneira, podemos concluir que com dados reais, o modelo até vai errar um pouco mais que nos testes, entretanto, a magnitude destes erros será levemente menor. Portanto, o modelo está bem sólido, dadas as estatísticas de um time da NFL, consegue prever a quantidade de vitórias com uma margem de erro de apenas 1 vitória.

	rs=41	rs=42	rs=43	rs=44	rs=45	média	melhoria
randomForest - Sem otimização - Conjunto de Treino							
mae	0,0683	0,0709	0,0672	0,071	0,0692	0,0693	
mse	0,0081	0,0086	0,008	0,0085	0,0081	0,0083	
randomForest - Otimizada - Conjunto de Treino							
mae	0,0673	0,0701	0,0666	0,0708	0,0689	0,0687	
mse	0,0078	0,0082	0,0077	0,0083	0,008	0,008	
randomForest - Otimizada - Conjunto de Teste							
mae	0,0727	0,0686	0,0716	0,0623	0,0697	0,0690	-0,02%
mse	0,0086	0,0079	0,0083	0,0069	0,008	0,0079	0,01%

8. Interpretação do Modelo Final

Como vimos na seção anterior, nosso melhor modelo consiste na floresta aleatória. Assim vamos tentar interpretar os resultados que ele nos retornou. Poderíamos pensar que *random forest*, por ser um modelo *ensemble*, seria uma caixa-preta difícil de interpretar. No entanto, esse modelo é baseado em árvores de decisão e, por isso, é naturalmente bem interpretável (com critérios de decisão bem definidos para cada atributo).

Por esses motivos, optamos por usar o atributo nativo de florestas aleatórias do *Scikit-Learn* chamado *feature_importances_*. Essa função calcula a importância de cada variável baseando-se na redução média de impureza (como o Erro Quadrático Médio, como estamos falando de regressão). Assim, montamos um gráfico que indica a relevância de cada atributo para o número de vitórias (atributo alvo) retornado pelo modelo:



9. Conclusão

No geral podemos dizer que obtivemos um modelo final/otimizado um tanto quanto preciso, com a floresta aleatória sendo capaz de prever a porcentagem de vitórias de um time com um jogo de precisão, o que consideramos como sendo muito bom. Aplicar os algoritmos de Aprendizado de Máquina aprendidos na disciplina em um problema pelo qual temos tanto apreço foi uma experiência enriquecedora no nível acadêmico e lúdico, o que contribui para o que consideramos como sucesso do trabalho final.

Pensando em melhorias futuras, poderíamos adaptar o modelo para prever a porcentagem de vitórias de um time ao longo de uma temporada. Por exemplo, treinar o modelo com os dados de apenas as seis primeiras partidas de cada temporada (o primeiro terço, aproximadamente) para determinar o *record* de um time ao final da temporada. Isso poderia inclusive ajudar a determinar quais times vão se classificar para a pós-temporada meses antes de isso ser concretizado.

Ademais, poderíamos aplicar os algoritmos de aprendizado não-supervisionado, em especial regras de associação e algoritmo Apriori, para visualizar informações que percebemos ao longo de um jogo. Por exemplo, um time que passa mais a bola tende a sofrer mais interceptações, outro que corre mais com a bola tende a ter mais posse de bola, assim por diante.

10. Fontes de pesquisa

INTERNET MOVIE DATABASE (IMDb). *Moneyball*. Disponível em: [IMDb](#). Acesso em: 12 maio 2026.

[University of Vienna – Institute of Sport Science – Computer Science in Sport](#). Viena: University of Vienna, [s.d.]. Disponível em: [página oficial](#). Acesso em: 12 maio 2026.

Richard Moss. The math behind Moneyball. [Medium](#), [s.l.], [s.d.]. Disponível em: [artigo completo](#). Acesso em: 12 maio 2026.

[Carnegie Mellon University – Department of Statistics & Data Science – Capstone Research Team 6](#). Carnegie Mellon University, 2024. Disponível em: [página do projeto](#). Acesso em: 12 maio 2026.